

Context-Aware Exploration in Reinforcement Learning Using Large Language Models

Supplementary Material

This document provides supplementary material accompanying our main paper accepted at ICTAI, 2026.

Contents

S1 Experiments and Environment Details	2
S1.1 Environment details	2
S1.2 Experimental setup	2
S2 Pseudocode and LLM-DQN Prompts	4
S2.1 LLM Inference Cost	13
S3 Ablation Studies	14
S3.1 Effect of Coverage information	14
S3.2 Effect of LLM’s state-dependent guidance	14

S1 Experiments and Environment Details

S1.1 Environment details

This section provides details about the benchmark environments used in our experiments. We evaluate the proposed approach on a diverse set of reinforcement learning tasks.

CartPole-v1: The CartPole environment [4, 5] is a classic control problem where a pole is attached to a cart moving along a one-dimensional track. The objective is to apply forces to the cart so that the pole remains upright for as long as possible. In the CartPole-v1 setting, an episode terminates after 500 time steps.

MountainCar-v0: The MountainCar environment [5] is a classical reinforcement learning problem where an underpowered car must drive up a steep mountain. Because the engine power is insufficient to directly climb the hill, the agent must first move away from the goal to build momentum before attempting to reach the top. The goal is to reach the flag located at the top of the mountain. This environment requires strategic exploration because the optimal solution initially moves away from the target. In our experiments, we adopt a modified reward formulation where successful completion of the task corresponds to a return threshold of +150, which is used as the convergence criterion during evaluation.

LunarLander: The LunarLander environment [5] models a two-dimensional landing problem where a spacecraft must safely land on a designated landing pad. The reward function encourages stable landings near the target location while penalizing crashes and excessive fuel usage. LunarLander is a more complex control task that requires coordinated thrust control and stable descent dynamics.

CliffWalking (Deterministic and Stochastic): The CliffWalking environment [4, 5] is a grid-world navigation task where the agent must move from a start state to a goal state while avoiding falling off a cliff. The environment is typically represented as a two-dimensional grid in which stepping into the cliff results in a large negative reward and resets the agent to the start position. The deterministic version transitions exactly according to the chosen action, while the stochastic variant introduces randomness in action execution, making the navigation task more challenging. This environment is commonly used to evaluate exploration strategies and risk-aware policies.

FrozenLake (Deterministic and Stochastic): The FrozenLake environment [5] is another grid-world navigation problem where the agent must traverse a frozen surface to reach a goal location while avoiding holes in the ice. In the deterministic variant, actions result in predictable transitions, whereas the stochastic version introduces slippage, causing the agent to move in unintended directions with some probability. This environment evaluates the agent’s ability to handle uncertainty and delayed rewards in discrete decision-making tasks.

S1.2 Experimental setup

All experiments were conducted using the OpenAI Gymnasium RL framework [5], which provides a standardized suite of environments for benchmarking reinforcement learning algorithms across diverse control and decision-making tasks. In all the tasks, agents interact with the environment by observing the current state, selecting an action, receiving a reward, and transitioning to a new state until the episode terminates. To evaluate our proposed LLM-guided exploration approach,

experiments were conducted across seven widely used RL environments discussed in section S1: Cart Pole, Mountain Car, Lunar Lander, Cliff Walking (deterministic and stochastic), and Frozen Lake (deterministic and stochastic), covering both control and navigation tasks.

Baseline Comparison Methods We compare the proposed LLM-DQN framework with two baseline methods: Vanilla DQN [3] and the state-of-art LLM-Explorer [2]. To ensure a fair comparison between our method and the baselines, all approaches were evaluated across 10 different random seeds for all seven environments. GPT-4 [1] models were used for our experiments. Additionally, the DQN hyperparameters were kept identical for both the proposed method and the baseline methods. The detailed hyperparameter settings for all environments are provided in Table S1.

Vanilla DQN [3]: Deep Q-Network (DQN) is a widely used RL algorithm that combines Q-learning with deep neural networks to approximate the Q-values.

LLM-Explorer [2]: LLM-Explorer periodically prompts an LLM with a task description and recent trajectory summaries to generate a state-independent exploration policy. Our approach differs in two key aspects: it constructs a richer context capturing the agent’s learning process and generates state-dependent exploration policies tailored to the current state.

Evaluation Metrics To evaluate learning performance across environments, we employ two complementary metrics based on the average episodic return: *Earliest Convergence* and *Convergence Rate %*. During training, the agent is evaluated after every episode using the current policy learned by the deep Q-network. The episodic returns obtained over these evaluation episodes are then averaged to compute the evaluation return for the corresponding training episode.

A run is considered to have converged when its average evaluation return reaches a predefined environment-specific threshold. For each converged run, *Earliest Convergence* denotes the earliest training episode at which this criterion is satisfied. This metric reflects the sample efficiency of the learning process, with a lower value indicating that the agent learns a successful policy with fewer interactions. Since an algorithm may not converge in every run, we additionally report *Convergence Rate %*, defined as the percentage of independent runs that successfully satisfy the convergence criterion. The *Earliest Convergence* is computed only over the successfully converged runs. Together, these two metrics capture complementary aspects of learning performance: *Earliest Convergence* measures how quickly an algorithm learns when it succeeds, while *Convergence Rate %* measures how consistently it succeeds across independent runs.

Table S1: Hyperparameter settings for each environment: k (LLM update frequency), N_{first} (episodes before first LLM call), N_{eval} (evaluation episodes), m (number of trajectories), p (past policies retained) and R_{th} (Return threshold).

Environment	k	N_{first}	N_{eval}	m	p	R_{th}
Mountain Car	5	5	10	3	10	150
Cart Pole	20	40	50	0	10	450
Lunar Lander	20	40	10	10	10	150
Cliff Walking (deterministic)	10	10	20	10	10	-13
Cliff Walking (stochastic)	10	10	20	10	10	-70
Frozen Lake (deterministic)	5	5	10	5	5	1
Frozen Lake (stochastic)	5	5	10	10	5	0.6

S2 Pseudocode and LLM-DQN Prompts

The detailed pseudocode of the proposed LLM-DQN framework is provided in Algorithm 1. The proposed framework transforms exploration into a meta-reasoning process over trajectories, environment descriptions and coverage summary.

Algorithm 1 DQN with LLM-Guided Exploration (N : training episodes, k : LLM update frequency, N_{eval} : evaluation episodes, m : greedy evaluation trajectories, p : history size, D : environment description)

- 1: Initialize Q-network parameters θ
- 2: Initialize target network parameters $\theta^- \leftarrow \theta$
- 3: Initialize replay buffer $\mathcal{B}$
- 4: Initialize history $\mathcal{H}_p \leftarrow \emptyset$
- 5: Initialize exploration policy $\pi_{\text{LLM}} \leftarrow \emptyset$
- 6: **for** episode $e = 1$ to N **do**
- 7: **if** $e \bmod k = 0$ **then**
- 8: Evaluate current greedy DQN policy:

$$\hat{J}(\pi_{\text{DQN}}) = \frac{1}{N_{\text{eval}}} \sum_{i=1}^{N_{\text{eval}}} \sum_{t=0}^T r_t^{(i)}$$

- 9: Collect m greedy evaluation trajectories $\mathcal{T}_m$
- 10: Compute coverage statistics $\mathcal{C}$ from $\mathcal{T}_m$ using the most recent LLM policy
- 11: Construct LLM input:

$$x = \left(D, \hat{J}(\pi_{\text{DQN}}), \mathcal{T}_m, \mathcal{C}, \mathcal{H}_p \right)$$

- 12: Query LLM:

$$\pi_{\text{LLM}} \leftarrow \Phi_{\text{LLM}}(x)$$

- 13: Update history:

$$\mathcal{H}_p \leftarrow \text{truncate}_p \left(\mathcal{H}_p \cup \{(\pi_{\text{LLM}}, \hat{J}(\pi_{\text{DQN}}))\} \right)$$

- 14: **end if**
- 15: Initialize state s_0
- 16: **for** each step t in episode **do**
- 17: Select action:

$$a_t = \begin{cases} \pi_{\text{LLM}}(s_t), & \text{during exploration,} \\ \arg \max_a Q_\theta(s_t, a), & \text{during exploitation.} \end{cases}$$

- 18: Execute a_t and observe (r_t, s_{t+1})
- 19: Store transition (s_t, a_t, r_t, s_{t+1}) in $\mathcal{B}$
- 20: Sample mini-batch from $\mathcal{B}$
- 21: Update θ by minimizing the TD loss:

$$\mathcal{L}(\theta) = \mathbb{E}_{(s,a,r,s')} \left[\left(r + \gamma \max_{a'} Q_{\theta^-}(s', a') - Q_\theta(s, a) \right)^2 \right]$$

- 22: Periodically update target network:

$$\theta^- \leftarrow \theta$$

- 23: **end for**
 - 24: **end for**
-

LLM-DQN Prompt for FrozenLake (Deterministic) Environment

%D: Environment description

You are an expert explorer helping an agent with exploration on the FrozenLake-v1 (4×4) environment. The agent moves using four actions (LEFT, DOWN, RIGHT, UP) over 16 discrete states. The objective is to reach the goal state while avoiding holes. Improve the exploration policy using the provided trajectories, coverage statistics, and previous policy performance. Preserve actions that consistently lead toward the goal and modify only actions that repeatedly lead to holes or unsuccessful trajectories.

% $\hat{J}(\pi_{\text{DQN}})$: Empirical evaluation of the current DQN policy

Average Evaluation Return: {avg_eval_reward:.2f}

Exploration Rate (ϵ): {current_epsilon:.2f}

%C: Coverage statistics of the current LLM exploration policy

Coverage Statistics:

LEFT Region: {coverage_info["left_pct"]:.2f}%

DOWN Region: {coverage_info["down_pct"]:.2f}%

RIGHT Region: {coverage_info["right_pct"]:.2f}%

UP Region: {coverage_info["up_pct"]:.2f}%

% $\mathcal{H}_p$: Previous LLM policies and their evaluation returns

History of Previous Policies and Evaluation Returns:

{past_policies.json}

% $\mathcal{T}_m$: Representative greedy evaluation trajectories

Representative Greedy Evaluation Trajectories:

{history.json}

Task: Analyze the representative trajectories, coverage statistics, and previous policy performance. Preserve actions that consistently appear in successful trajectories and modify only actions corresponding to states that repeatedly lead to holes or unsuccessful trajectories. Improve the evaluation return while making as few policy changes as possible.

%Formatting Instructions

Output JSON only in the following format:

```
{
  "policy": [16 integers in [0,3]],
  "reasoning": "string"
}
```

LLM-DQN Prompt for Cart Pole

%D: Environment description

You are a smart environment explorer working on the Cart Pole environment. The goal of the agent is to balance a pole on a cart. However, your goal is to generate a rule-based exploration policy to help the DQN agent learn faster. The observation space is 4-dimensional: x (cart position), v (cart velocity), θ (pole angle), ω (pole angular velocity). The action space is discrete: 0 (push cart left), 1 (push cart right).

% $\hat{J}(\pi_{\text{DQN}})$: Empirical evaluation of the current DQN policy

Average Evaluation Return (most recent policy, evaluated over 50 episodes): {eval_r_for_llm:.2f}

%C: Coverage statistics of the current LLM exploration policy

Coverage Statistics (per rule-defined region):

{coverage_info}

% $\mathcal{H}_p$: Previous LLM policies and their evaluation returns

History of Previous Policies, Evaluation Returns, and Coverage Statistics:

{history_text}

Task:

Use both the evaluation return and the region coverage statistics to generate the next exploratory policy. The objective is not exploration itself, but to improve the DQN agent's learning performance and achieve higher evaluation returns. Rules must be non-conflicting: use mutually exclusive conditions, or rely on ordering, where the first matching rule applies.

%Formatting Instructions

Output JSON only in the following format:

```
{
  "policy_type": "rules",
  "name": "policy_ep_{episode}",
  "rules": [
    {"if": [ {"feature": "theta", "op": "<", "value": -0.1}, ... ], "then": 1},
    ...
  ],
  "default_action": 0,
  "notes": "string"
}
```

LLM-DQN Prompt for Lunar Lander

%D: Environment description

You are a smart environment explorer working on the Lunar Lander environment, where the agent must safely land the spacecraft between the flags while staying stable during descent. Your role is to generate a rule-based exploration policy that helps the DQN agent learn faster. The observation space is 8-dimensional: x, y (position), v_x, v_y (velocity), θ (angle), ω (angular velocity), and left/right leg contact. The four actions are: 0 (do nothing), 1 (fire left engine), 2 (fire main engine), 3 (fire right engine). The environment rewards proximity to the pad, low speed, low tilt, and leg contact, while penalizing engine use, tilt, and speed, with a bonus of +100 for a safe landing or a penalty of -100 for a crash. A successful landing is typically marked by x, v_x, v_y, θ , and ω all near 0, with at least one leg touching down.

% $\hat{J}(\pi_{\text{DQN}})$: Empirical evaluation of the current DQN policy

Average Evaluation Return (of the most recent suggested policy, over 10 greedy evaluation episodes): {eval_r_for_llm:2f}

%C: Coverage statistics of the current LLM exploration policy

Coverage statistics indicate the percentage of visited states belonging to each rule-defined region during the 10 evaluation episodes:

{coverage_info}

% $\mathcal{H}_p$: Previous LLM policies and their evaluation returns

For each previous policy, its rules, evaluated return, and coverage statistics are provided:

{history_text}

% $\mathcal{T}_m$: Representative greedy evaluation trajectories

Each trajectory is summarized by sampling states from beginning and middle of the episode, while preserving the final states to capture terminal behavior (successful landing or crash).

{trajectory_text}

Task: Use the evaluation returns, coverage statistics, and trajectories together, treating trajectories as the primary evidence for deciding which rules to modify. Preserve rules from successful or high-return policies, modify only the smallest number of rules (preferably one or two) unless trajectories reveal a new repeated failure pattern, and reuse successful rules from previous policies where appropriate.

%Formatting Instructions

Output JSON only:

```
{
  "policy_type": "rules",
  "name": "policy_ep_{episode}",
  "rules": [
    {"if": [ {"feature": "vy", "op": "<", "value": -0.5}, ... ], "then": 1},
    ...
  ],
  "default_action": 0,
  "notes": "string"
}
```

LLM-DQN Prompt for MountainCar

%D: Environment description

You are controlling a car in Mountain Car environment. The goal is to reach the flag at position > 0.5 . Actions: 0 (left), 1 (nothing), 2 (right). Gravity pulls the car down, and the engine is too weak to climb directly, so the agent must build momentum by rocking back and forth. The task is to define a non-conflicting policy that splits the state space (velocity, position) into a Low Energy state, which needs momentum building, and a High Energy state, which should drive up the hill.

% $\hat{J}(\pi_{\text{DQN}})$: Empirical evaluation of the current DQN policy

Eval Score (Greedy): {eval_score:.2f}

Exploration Rate (ϵ): {current_epsilon:.2f}

%C: Coverage statistics of the current LLM exploration policy

Low Energy Region: {coverage_info["low_energy"]} %

High Energy Region: {coverage_info["high_energy"]} %

% $\mathcal{H}_p$: Previous LLM policies and their evaluation returns

History of Policies (each with its splits, average return, epsilon, and coverage statistics):
{history_json}

% $\mathcal{T}_m$: Representative evaluation trajectories

Train Trajectories (Exploration), summarized as return and length per run:
{traj_summary}

Task:

Use both the evaluation score and the coverage statistics to update the state-space split. Provide new values for the velocity split, position split, and the actions assigned to the low- and high-energy regions.

%Formatting Instructions

Output JSON only in the following format:

```
{
  "vel_split": float,
  "pos_split": float,
  "action_low_energy": int,
  "action_high_energy": int,
  "reasoning": "string"
}
```

LLM-DQN Prompt for FrozenLake(Stochastic) Environment

%D: Environment description

You are an expert explorer helping an agent with exploration on FrozenLake-v1 (4×4 grid), where the agent can take action 0, 1, 2, or 3, over 16 discrete states arranged as:

```
S F F F
F H F H
F F F H
H F F G
```

S is the start, F is frozen surface, H is a hole (episode ends, 0 reward). The environment is slippery: each action results in one of three neighboring directions with equal probability, rather than the intended direction. Action mapping: 0 = LEFT, 1 = DOWN, 2 = RIGHT, 3 = UP. The task is to propose a policy that takes safe actions to maximize the average return.

% $\hat{J}(\pi_{\text{DQN}})$: Empirical evaluation of the current DQN policy

Current Average Evaluation Return: {avg_eval_reward:.2f}

Exploration Rate (ϵ): {current_epsilon:.2f}

%C: Coverage statistics of the current LLM exploration policy

Coverage Statistics:

LEFT Region: {coverage_info["left_pct"]:.2f}%

DOWN Region: {coverage_info["down_pct"]:.2f}%

RIGHT Region: {coverage_info["right_pct"]:.2f}%

UP Region: {coverage_info["up_pct"]:.2f}%

% $\mathcal{H}_p$: Previous LLM policies and their evaluation returns

{past_policies_json}

% $\mathcal{T}_m$: Representative greedy evaluation trajectories

{history_json}

Task: The trajectories are the primary source of evidence. Preserve actions that consistently appear in successful trajectories; modify only actions with repeated evidence of failure. Preserve the policy with minimal changes if it improves on the previous return, and correct only the failing states if it degrades.

%Formatting Instructions

Output JSON only:

```
{
  "policy": [16 integers in [0,3]],
  "reasoning": "string"
}
```

LLM-DQN Prompt for CliffWalking(Deterministic) Environment

%D: Environment description

You are an expert explorer helping an agent with exploration on CliffWalking-v1 (is_slippery=False). The environment is deterministic: every action always produces the intended movement. The grid is 4×12 , giving states 0–47. Rewards are -1 per step and -100 for entering the cliff (states 37–46), after which the agent resets to the start state 36. The agent moves using four actions (0: UP, 1: RIGHT, 2: DOWN, 3: LEFT). Do not enter cliff states; focus especially on cliff states (37–46), states near the cliff (25–35), and the start state (36). Improve the exploration policy using the provided trajectories, coverage statistics, and previous policy performance.

% $\hat{J}(\pi_{\text{DQN}})$: Empirical evaluation of the current DQN policy

Average Evaluation Return: {avg_eval_reward:.2f}

Exploration Rate (ϵ): {current_epsilon:.2f}

%C: Coverage statistics of the current LLM exploration policy

Coverage statistics indicate the percentage of states visited during the {n_trajectories_llm} greedy evaluation trajectories that fall under each action region defined by the current exploration policy.

Action {region}: {pct:.2f}% (repeated for each action present in the current partial policy)

% $\mathcal{H}_p$: Previous LLM policies and their evaluation returns

History of Previous Policies and Evaluation Returns:

{past_policies.json}

% $\mathcal{T}_m$: Representative greedy evaluation trajectories

Representative Greedy Evaluation Trajectories:

{history.json}

Task:

Identify where the agent is repeatedly failing or getting stuck, focusing especially on states near the cliff. Improve only the critical states and avoid repeating unsuccessful policy modifications; if similar states were suggested previously without improving the evaluation return, modify different states instead. The objective is to reach the goal using the shortest safe path while avoiding the cliff and avoiding loops or oscillations between states. Prefer moving RIGHT toward the goal whenever safe, and use UP only when necessary to avoid the cliff. Output only a partial policy: do not assign actions to all states, and do not assign the same action to all states; modify only 2–5 of the most problematic states, and keep the partial policy as small as possible.

%Formatting Instructions

Output JSON only in the following format:

```
{
  "partial_policy": {state: action},
  "reasoning": "string"
}
```

LLM-DQN Prompt for CliffWalking(Stochastic) Environment

%D: Environment description

You are an expert explorer helping an agent with exploration on CliffWalking-v1 (is_slippery=True). The grid is 4×12 , giving states 0–47. Rewards are -1 per step and -100 for entering the cliff (states 37–46), after which the agent resets to the start state 36. Since the environment is stochastic, each action has a probability of sliding into one of three possible directions rather than always producing the intended movement (e.g., action 0 may result in left, up, or right; action 1 in up, right, or down; action 2 in right, down, or left; action 3 in down, left, or up). Never enter cliff states; focus especially on cliff states (37–46), states near the cliff (25–35), and the start state (36). Identify failure patterns from trajectories, avoid the cliff aggressively, and prefer safe paths. Improve only critical states.

% $\hat{J}(\pi_{\text{DQN}})$: Empirical evaluation of the current DQN policy

Average Evaluation Return: {avg_eval_reward:.2f}

Exploration Rate (ϵ): {current_epsilon:.2f}

%C: Coverage statistics of the current LLM exploration policy

Coverage statistics indicate the percentage of states visited during the 10 greedy evaluation trajectories that fall under each action region defined by the current exploration policy.

Action {region}: {pct:.2f}% (repeated for each action present in the current partial policy)

% $\mathcal{H}_p$: Previous LLM policies and their evaluation returns

Past Policies and their performance:

{past_policies.json}

% $\mathcal{T}_m$: Representative greedy evaluation trajectories

Current Trajectories of DQN Learning:

{history.json}

Task:

Identify where the agent is failing, focusing especially on states near the cliff (35–46). Improve only critical states, and avoid repeating bad policies; if similar states were already suggested in past policies and did not improve returns, choose different states instead. The goal is located at the far right of the grid; the agent should prioritize reaching the goal in the minimum number of steps while avoiding loops or oscillations between states. Prefer moving RIGHT unless it leads directly into a cliff. Use UP only to escape danger, not as a default action. Do not assign the same action (e.g., LEFT or UP) to all states.

%Formatting Instructions

Output JSON only in the following format:

```
{
  "partial_policy": {state: action},
  "reasoning": "string"
}
```

Table S2: LLM inference cost of the proposed method across benchmark environments [5].

Environment	Avg. LLM Calls/Run	Cost (10 Seeds) (\$)
MountainCar	1.1	0.05
CartPole	11.7	2.79
LunarLander	11.0	0.54
CliffWalking (Det.)	3.2	0.01
CliffWalking (Sto.)	7.3	0.04
FrozenLake (Det.)	8.6	1.93
FrozenLake (Sto.)	26.2	3.64
Average / Total	9.87	9.00

S2.1 LLM Inference Cost

Table S2 summarizes the average number of LLM calls per training run and the estimated inference cost across 10 seeds for each environment. Overall, the proposed approach required an average of 9.87 LLM calls per run, with an estimated total inference cost of approximately \$9 across all seven benchmark environments.

Table S3: Coverage ablation for LLM-DQN. Results show the Earliest Convergence (EC) episode mean $\pm$ standard deviation and Convergence Rate (CR) over 10 runs; lower values indicate faster convergence.

Environment	LLM-DQN (Ours)		LLM-DQN w/o Coverage	
	EC	CR (%)	EC	CR (%)
Mountain Car	7.3 ± 2.6	100	7.8 ± 2.27	100
Cart Pole	118.0 ± 51.7	60	88.4 ± 14.58	50
Lunar Lander	219.4 ± 64.2	100	315.0 ± 109.26	80
Cliff Walking (deterministic)	37.5 ± 14.1	100	124.0 ± 127.28	20
Cliff Walking (stochastic)	75.9 ± 20.1	100	94.7 ± 30.6	100
Frozen Lake (deterministic)	45.3 ± 27.7	100	49.6 ± 24.58	100
Frozen Lake (stochastic)	82.71 ± 30.57	70	114.25 ± 63.35	80

S3 Ablation Studies

We conduct two ablation studies to evaluate the contributions of the two key design components of the proposed framework: (i). the inclusion of coverage information in the LLM context, and (ii). the generation of state-dependent exploration policies. In the first ablation, we evaluate the effect of the proposed coverage summary by comparing the performance of the framework with and without the coverage information in the LLM prompt.

In the second ablation, we augment LLM-Explorer [2] with the proposed coverage summary while retaining its state-independent exploration strategy. This isolates the contribution of state-dependent exploration policies by attributing the remaining performance difference to the ability of the proposed framework to generate state-specific exploration guidance.

S3.1 Effect of Coverage information

Table S3 shows that incorporating coverage generally improves convergence speed while maintaining comparable convergence rates. By identifying underexplored state regions, the coverage summary enables the LLM to refine its exploration policy more effectively.

S3.2 Effect of LLM’s state-dependent guidance

Table S4 isolates the effect of state conditioning and shows that state-dependent LLM guidance achieves faster convergence than the state-independent variant, with similar convergence rates. This indicates that state-dependent guidance is able to exploit coverage feedback more effectively, resulting in more informed exploration decisions.

Table S4: Ablation study comparing LLM-guided state-dependent (Ours) and state-independent exploration, with coverage information used in both settings. Results report the Earliest Convergence (EC) episode mean $\pm$ standard deviation and Convergence Rate (CR) over 10 runs; lower values indicate faster convergence.

Environment	LLM-DQN (Ours)		State-Independent LLM Exploration	
	EC	CR (%)	EC	CR (%)
Mountain Car	7.3 ± 2.6	100	— — $\pm$ — —	0
Cart Pole	118.0 ± 51.7	60	229.57 ± 130.55	70
Lunar Lander	219.4 ± 64.2	100	210.4 ± 80.60	100
Cliff Walking (deterministic)	37.5 ± 14.1	100	25.3 ± 7.0	100
Cliff Walking (stochastic)	75.9 ± 20.1	100	104.20 ± 50.65	100
Frozen Lake (deterministic)	45.3 ± 27.7	100	54.7 ± 16.67	100
Frozen Lake (stochastic)	82.71 ± 30.57	70	102.20 ± 34.96	100

References

- [1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altschmidt, Sam Altman, Shyamal Anadkat, et al. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.
- [2] Qianyu Hao, Yiwen Song, Qingmin Liao, Jian Yuan, and Yong Li. Llm-explorer: A plugin reinforcement learning policy exploration enhancement driven by large language models. *Advances in Neural Information Processing Systems*, 38:93586–93627, 2026.
- [3] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, Stig Petersen, Charles Beattie, Amir Sadik, Ioannis Antonoglou, Helen King, Dhharshan Kumaran, Daan Wierstra, Shane Legg, and Demis Hassabis. Human-level control through deep reinforcement learning. *nature*, 518(7540):529–533, 2015.
- [4] Richard S Sutton and Andrew G Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2018.
- [5] Mark Towers, Ariel Kwiatkowski, Jordan Terry, John U. Balis, Gianluca De Cola, Tristan Deleu, Manuel Goulão, Andreas Kallinteris, Markus Krimmel, Arjun KG, Rodrigo Perez-Vicente, Andrea Pierré, Sander Schulhoff, Jun Jet Tai, Hannah Tan, and Omar G. Younis. Gymnasium: A standard interface for reinforcement learning environments, 2025.